School of Computing, Engineering, and Physical Sciences
Assessment Guidance Coversheet

Module Code:	COMP10037	
Module Title:	Games Console Programming	
Module Co-ordinator:	Dr. Paul Keir	
Assessment set by:	Dr. Paul Keir	
Learning Outcomes Assessed:	L1 Demonstrate a critical appreciation of the hardware architectures of modern games console platforms. L2 Apply design techniques and optimization strategies for developing games software for a modern games console platform using a low-level API. L3 Demonstrate a critical appreciation of the differences between coding for general purpose platforms (e.g. PC) and fixed hardware platforms with limited OS services. L4 Demonstrate critical awareness of the issues surrounding parallel processing, and its use on modern games console platforms.	
Issued:	Tuesday, November 10th, 2026	
Deadline:	5pm, Friday, December 4th, 2026	
Feedback:	Friday, December 25th, 2026	
Individual / Group assessment:	Individual ❑	Group ☑

Anonymity

The University Regulations makes it clear that all assessments are marked anonymously (Regulation 3.4) unless the assessment itself renders anonymity impossible e.g. placements, presentations, practical assessments. Unless otherwise stated, do not put your name or Banner ID on your submission.

Referencing

The standard referencing style at UWS is **Cite Them Right (CTR) Harvard** and your references should be in accordance with these guidelines. The University provides a drop-down menu that lets you see examples of how you should reference journals, books, websites etc. Guidance on referencing styles can be found on the UWS Library website: https://uws-uk.libguides.com/referencing.

Extenuating Circumstances

The University recognises that, from time to time, you may encounter issues which may prevent you from being able to submit or undertake an assessment. Where this is the case, you can complete an Extenuating Circumstances Submission (ECS) for consideration. The ECS will be forwarded to the School Assessment Board to take account of this declaration in recording your module marks[1]. Guidance on ECS

[1]Note that any mark / grade you receive is provisional and may be subject to change until there has been internal

claims can be found: https://www.uws.ac.uk/current-students/supporting-your-studies/exams-assessment-appeals/academic-appeals-extenuating-circumstances.

Generative AI

The type of Generative AI you are allowed to use within your assessment is:

Type	Description	Allowed
1	**Restricted** Only the use of routine and established tools, such as auto-transcription, spell checkers, grammar check is permitted.	☑
2	**Specified** Generative AI may be used for clearly delineated tasks as appropriate / allowed / recommended, although its use is **not** mandatory in order to complete the assessment.	❑
3	**Open** No specific restrictions but with requirement to track key stages / tools utilised. The use of such tools is **not** mandatory in order to complete the assessment. This may include: (i) Socratic chatbot, (ii) summarisation, simplification, synthesis, and translation, (iii) generative illustrative media content, and (iv) support production of multimedia artefacts.	❑
4	**Embedded** Generative AI is a feature of the assessment itself. Here the use of Generative AI is a focal aspect of the assessment. This may include (i) using named tools for specific outcome, (ii) output comparison and critique, and (iii) develop or error check code.	❑

Assessment Student Declaration Coversheet

All students are expected to complete the Student Declaration and insert that as the first page of your submission. The Module Co-ordinator will have uploaded this to Aula for you to complete.

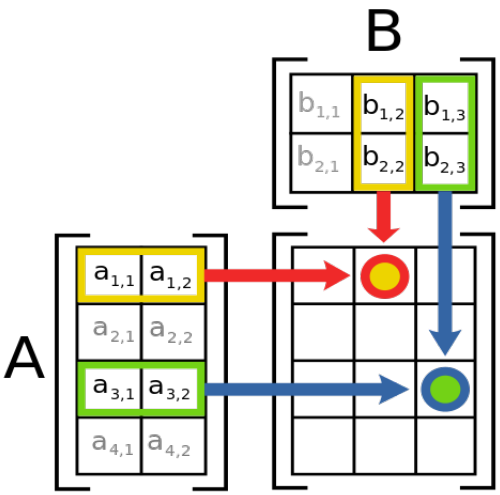
- For written assessments, insert the Declaration as the first page of your document.
- For assessments not in the written format (e.g. video, audio, presentation, or practical work), submit the Declaration as a separate file.
- For group assessments, unless otherwise directed by your lecturer, the group may submit a single shared declaration.

moderation, external examination and ratification at the School Assessment Board.

Second Assignment
Games Console Programming
COMP10037

Issue Date: Tuesday, November 10th, 2026
Due Date: **5pm, Friday, December 4th, 2026**

Benchmarking Matrix Multiplication



In this assignment you will extend the serial matrix multiplication code provided, to create a parallel version which operates on matrices of arbitrary size (i.e. *much* larger than 8x8). You should build on your knowledge of SIMD parallelism; and multithreading. The code you develop can be created for testing in a cross-platform manner; allowing you to develop your code both on the PS5 devkit, as well as a common desktop computer.

You should work in your allocated team or individually as arranged. Submit the code you develop (including CMakeLists.txt), along with a short PDF report of approximately 1000 words; as a single zip to Brightspace by the date noted

above. Please do not submit the contents of your CMake "build" directory ... simply delete it before submission.

Methodology

You are provided with a C++ file which multiplies two square matrices. The code is configured to multiply a simple matrix by itself. When you have implemented your parallel matrix multiplication, you can check your result with the serial version using `mat::operator==` via an expression such as `mres==m`. An `assert` call can help check this.

You should then create one or more parallel versions using the methods introduced in the class. That is, you should create either a multithreaded, or a SIMD version. You can do both, though the marking scheme recognises a second version as an extra component for ambitious teams.

Refer to the slides from week's #5, #6, #8 and #9 and corresponding lab materials, to produce a version which works with matrix elements which are either single-precision `float`s or double-precision `double`s. These will use `__m128` and `__m256d` respectively; and intrinsics including `_mm_mul_ps` and `_mm256_mul_pd`.

Also create a shared-memory parallel version using the C++ Standard Threading Library. You may want to assign each core the task of calculating a contiguous set of rows in the output matrix (i.e. static partitioning). For example, with a pair of 16×16 matrices, and 8 threads, the first thread would calculate the first two rows of the result (i.e. $16 \div 8$ rows).

This should ideally work generically on single or double precision data, and upon arbitrary matrix sizes (order). To start with, you may wish to focus on an 8×8 matrix for simplicity (but this is still far too small for serious benchmarking).

You should finally vary the size of the matrices, and benchmark performance. You can prepare your benchmarking on your own machine at home, but the benchmarks included in your report should include timings on the PS5.

Two variables should be shown on each line graph: think of 1, 2, 4, 8, 16 threads on the x-axis, and duration (time) on the y-axis, with the largest data size; or data sizes from `(1 << 6) to (1 << 10)` on the x-axis, and duration (time) on the y-axis. Many other variations and combinations can be presented, including varying SIMD widths.

Remember, for each set of parameters, you should not only obtain the average (mean) time of your (say 6) benchmark runs ... you should also calculate the standard deviation, and include these as error bars on your graphs.

Originally the size of the matrices is defined by the `unsigned` value `SZ`. Alternatively, you should plan to configure all matrix sizes, and thread counts, SIMD widths etc. within your C++ program. Ensure you describe how you ensure your calculations produce the correct results throughout.

Requirements

- Implement matrix multiplication in parallel.
- Benchmark how varying the array size from `(1 << 6) to (1 << 10)` affects performance.
- If you include multithreading, you should also benchmark with a varied number of threads/cores.
- Implement a double-precision version of your solution.
- Ensure you run each benchmark a few times (say 6). Take the average, and calculate the standard deviation.
- Create a report to explain your approach, and the feature set you provide. Include graphs to examine all aspects of performance.

Marking Scheme

The assignment is worth 30% of the marks awarded for the entire COMP10037 module. The following provides a breakdown of the marking scheme:

A parallel matrix multiplication implementation (either SIMD or Multithreading)	10
A parallel matrix multiplication implementation incorporating both SIMD *and* multithreading	3
A double-precision version (i.e. double rather than float)	2
Consistent checks that the results are correct (e.g. assert calls)	2
Tidy, well structured code	1
A comprehensive set of benchmark results (within the report)	3
1000 word report with graphs/figures	6
Average of your peer review mark from your team	3

Plagiarism

Ensure your work is your team's alone. You can discuss ideas with other teams, regarding how to prepare a solution, but the *copying or sharing of code is not permitted*.